DSA0309 – NATURAL LANGUAGE PROCESSING
ASSIGNMENT – 05

Language Generation and Discourse Analysis

Name: Yoga Madha A

Register Number:192425058

Department / Section:B.Tech AI&ML(3rd Year)

GitHub Link: https://github.com/YOGAMADHA/DSA0309-NLP/tree/main/Assignment%205

PART A – DESIGN, SPECIFICATION & PSEUDOCODE

A1 – Define the Discourse Analyst's World

A1.1 Position and responsibilities

Pipeline: raw report → sentence/token processing → discourse analysis (mentions, coreference, discourse units and relations) → question answering / discourse planning → language generation (content plan to surface text) → optional translation. The discourse module receives segmented text plus linguistic/context evidence. It must preserve the source propositions, entity identity, temporal order and uncertainty; it may add links, labels and structure but must not invent facts. The generation module receives a meaning representation and discourse plan. It must preserve meaning, commitments, reference and required order; it may change wording, aggregation and referring-expression form.

Design issues and ownership:

- Unit of analysis: this task begins with numbered sentences; a corpus annotation scheme fixes sentence boundaries, while the system designer may split clauses into EDUs.
- Knowledge source: agreement and syntax are observable linguistic evidence; salience and world knowledge are system features. Example: 'He' is compatible with Ravi by person/gender; 'it' is compatible with a singular inanimate server.
- Explicit/implicit coherence: 'because' explicitly signals Explanation; adjacency plus a causal reading may imply Explanation. The annotation scheme defines labels; the system designer chooses inference rules.
- Granularity: sentence-level RST-like units versus clause-level EDUs. The corpus scheme determines available gold granularity; the designer selects internal granularity when no annotation is supplied.
- Evaluation: link precision/recall/F1 and boundary F1. The task sheet fixes acceptable metric families; the designer defines markables and matching policy.
- Ambiguity: retain multiple candidates or mark unresolved rather than guessing. The corpus scheme should encode ambiguity if possible; confidence threshold and abstention policy are designer choices.

A1.2 Measurement model

A markable is a referring expression whose identity is to be resolved (proper name, definite noun phrase or referential pronoun); pleonastic 'it' is excluded. A predicted link is correct only if its anaphor and antecedent entity match the gold link exactly. Gold links = 4.

System	Correct	Proposed	Precision	Recall / F1
X	3	5	3/5 = 0.600	3/4 = 0.750; F1 = 0.667
Y	3	3	3/3 = 1.000	3/4 = 0.750; F1 = 0.857

Deploy Y: it has the same recall as X with fewer false positive links and higher F1. Both miss one gold link. This metric ignores bridging (e.g., 'module' related to 'server'), near-identity, partial spans, confidence and downstream damage. A single wrong link can change an answer such as which module was replaced even when aggregate F1 looks acceptable.

A1.3 Reference-resolution pseudocode

ALGORITHM RESOLVE(pronoun, sentence, discourse_state)
INPUT: pronoun p, sentence s, ordered entity memory M
OUTPUT: antecedent or UNRESOLVED / AMBIGUOUS / NONREFERENTIAL
1. IF p is pleonastic by construction/context (e.g., “It is raining”) THEN
    RETURN NONREFERENTIAL
2. C ← entities introduced before p in current discourse segment
3. FILTER C by number and person agreement; reject syntactically impossible candidates
4. Rank remaining candidates by:
    (a) subject/topic role, (b) recency, (c) repeated mentions,
    (d) parallel grammatical role with the previous clause
5. IF C is empty THEN RETURN UNRESOLVED
6. Score each candidate with fixed weighted sum; sort descending
7. IF top two scores tie THEN RETURN AMBIGUOUS with both candidates
8. RETURN highest-scoring candidate
9. Update entity memory with chosen link and mention position
END

The candidate set contains only entities introduced before the pronoun in the same discourse segment. Agreement is a hard filter; role, recency, repetition and parallelism are ranking features. A non-referential pronoun is removed before candidate search.

A1.4 Resolver trace

For the supplied T, the four gold links are: It in S2 → server_1; He in S4 → Ravi; it in S4 → server_1; The suite in S5 → suite_1. The last is a definite description, not a pronoun, and is included in the markable evaluation.

Sentence / expression	Candidates before resolution	Chosen link	State carried forward
S2 It	server_1 (recent, subject/topic); department (older); lab (older)	server_1	server_1 becomes current topic; old machine introduced as a new entity
S4 He	Ravi (recent subject); server_1 (recent object); toolchain (recent)	Ravi	Ravi promoted as subject/topic
S4 it	server_1 (recent object of configure); suite_1 (introduced in same sentence; incompatible with intended location role)	server_1	server_1 retained; suite_1 added to focus
S5 The suite	suite_1 (exact repeated head from S4)	suite_1	suite_1 becomes topic of crash clause

The tricky case is S2: a nearest-noun-only heuristic may select ‘lab’ or the immediately preceding noun phrase instead of the new server. The resolver uses number/agreement, syntactic role, discourse topic

and semantic plausibility. S4 ‘it’ is resolved to the configured server because it is the location of installation; a production system would encode semantic role constraints explicitly.

A2 – Discourse Units, Graph and Next-Mention

A2.1 Boundaries

Use one sentence as the initial unit because the task provides sentence segmentation. Start a new unit at each sentence boundary; additionally split a sentence only at a clause boundary when each clause expresses an independently assertable event. A paragraph break is a hard boundary. A topic/entity shift, time/place shift or explicit discourse connective is evidence for a relation or optional clause split, not by itself permission to cut inside a syntactic constituent.

Units: U1 = S1 (purchase/new server); U2 = S2 (replacement of old machine); U3 = S3 (Ravi configures server); U4 = S4 (Ravi installs suite); U5 = S5 (suite crashes); U6 = S6 (Ravi reports fault; vendor replaces module).

A2.2 Segmentation and discourse graph pseudocode

```
ALGORITHM BUILD_DISCOURSE_GRAPH(sentences)
1. Create one EDU for each input sentence; split coordinated clauses only
   when each clause has an independent predicate and proposition.
2. For each EDU u after the first:
   a. Detect explicit connective and map it to a relation label.
   b. Otherwise infer a relation from event/time/entity evidence;
      if evidence is insufficient, use UNDECIDED (do not force SEQUENCE).
   c. Choose nucleus (central claim) and satellite (supporting clause).
   d. Attach to the nearest open node on the right frontier that yields
      a licensed relation; otherwise attach to the document root.
   e. Add edge to successor list of parent and predecessor list of child.
3. Mark a node closed when its subtree is complete and it is no longer
   on the right frontier; do not attach new material to a closed node.
4. Return EDU nodes and labelled edges.
END
```

Graph for T (simplified):

```
ROOT
 ├─ U1: department bought new server
 │   └─ U2: it replaced old machine       [Elaboration / replacement detail]
 ├─ U3: Ravi configured server
 │   └─ U4: he installed compiler suite    [Sequence]
 └─ U5: suite crashed                      [Result of installation/build context]
    └─ U6: Ravi reported fault; vendor replaced module [Explanation/response]
```

Deepest attachment: U6 attached under U5 (depth 2 below ROOT).

For an explicit cue such as ‘because’, assign Explanation; without a cue, infer only when event and causal evidence support it; if a later sentence refers back to U1, attach it to U1 only if U1 remains on the right frontier. The structural restriction is the Right-Frontier Constraint: a new EDU may attach only to the current right frontier, not an already closed node.

A2.3 Backward next-mention scan

Assumption: an entity is available on exit if it remains on the right frontier and is mentioned later in the document; entities that never appear again are not carried. Distance is counted in EDU steps to the next mention.

Unit / mention	Entity	Available after unit?	Distance to next mention
U2: It	server_1	Yes	1 (U3)
U2: old machine	old_machine	No	No later mention
U3: Ravi	Ravi	Yes	1 (U4), then U6
U3: server	server_1	Yes	1 (U4)
U3: toolchain	toolchain	No	No later mention

A2.4 Design examination

- A connective in the middle of a sentence links clauses; it does not automatically create a sentence boundary. Cutting there can separate a subordinate clause from its governor and create a false EDU or broken dependency.
- A forward scan cannot know the next mention until it has already passed it; it therefore cannot annotate the current mention with future distance without storing and revisiting information.
- A discourse graph exposes central claims, support, cause and result. A summariser can select nuclei and omit satellites; a flat sentence list does not explicitly encode those roles.

A3 – Generator with Focus, Referring Expressions and Aggregation

A3.1 Information to remember

Maintain (i) a focus list of at most k=2 entities ordered by salience, (ii) a descriptor per entity containing last form, number, gender, last grammatical role and last mention position, (iii) an aggregation buffer for compatible propositions, and (iv) a given/new flag. Update descriptors after every realised mention; promote subjects/topics; evict the lowest-salience entity when full; clear the aggregation buffer at a paragraph boundary or incompatible relation.

A3.2 Form choice and aggregation pseudocode

ALGORITHM CHOOSE_FORM(entity e, focus F, descriptors D)
1. IF e is new THEN emit an indefinite NP (“a server”); mark NEW.
2. ELSE IF e is in F:
    IF exactly one compatible focused antecedent exists THEN emit pronoun;
    ELSE emit a definite description to avoid ambiguity.
3. ELSE IF |F| < k THEN emit definite description; add e to F.
4. ELSE evict lowest-salience entity; emit definite description; add e.
5. Update last form, number, gender, role and salience.
ALGORITHM AGGREGATE(p, q)

IF same subject AND compatible tense/time AND relation is Sequence/Result
  THEN combine with conjunction while preserving event order
ELSE emit separately.
Selection rule: use the shortest referring form that uniquely identifies
the intended entity within the current focus; otherwise use a noun phrase.

A3.3 Trace and word cost

A transparent cost estimate (using the task's 3-word full-name/description and 1-word pronoun costs; proposition content words are excluded from this referring-expression comparison):

Fact	Chosen referring form	Focus after fact	Aggregation / output plan	Reference words (running)
F1 memory(server_1,32GB)	a server (new)	[server_1]	The server has 32 GB memory.	3
F2 configure(Ravi,server_1)	Ravi (new); the server (known)	[Ravi, server_1]	Ravi configured the server.	6
F3 install(Ravi,suite_1,server_1)	Ravi; a compiler suite (new); the server	[Ravi, suite_1] (server evicted after use)	Ravi then installed a compiler suite on the server.	12
F4 crash(suite_1,during build)	the suite (focused)	[suite_1, Ravi]	The suite crashed during a build.	15

Naïve baseline: full noun phrases at every mention; approximate reference cost 3+3+3+3+3+3 = 18 words (counting six entity mentions). Proposed generator: 15 reference words in this bookkeeping; a careful human may use 'It' once and aggregate clauses, approximately 13 reference words. The saving is bought with more state, possible ambiguity, and the risk of over-aggregation. Exact surface word counts depend on the chosen wording.

A3.4 Failure case

Microplan: F1 introduce server_A; F2 introduce server_B; F3 install suite_A on it. With both servers in focus and same number/gender, a pronoun 'it' is ambiguous. The automatic generator may emit 'it' because an entity is focused. A hearer model plus semantic-role constraints (installation location) or a stronger centering transition would force 'the second server'.

A4 – Entity–Discourse Graph

A4.1 Diagnose weak parser

The adjacent-noun rule labels S1→S2 as ELABORATION because 'server' is shared, which may be acceptable, but it can wrongly label S2→S3 as ELABORATION merely because both contain a machine-related noun. It labels S3→S4 as ELABORATION from repeated 'Ravi/server', although the intended relation is Sequence/continuation. S4→S5 is treated as SEQUENCE if there is no shared noun under the simplistic matching rule, missing the causal/result relation: the installed suite crashes during a build. In S3→S4, 'it' can also be attached to the nearest noun 'toolchain' instead of server if resolution is purely nearest-noun.

A4.2 Entity graph design

ALGORITHM UPDATE_GRAPH(sentence s)
1. Detect noun phrases, names and pronouns; assign each mention a span.
2. For a name/description:
   a. Search existing referents using head, attributes, number, time and context.
   b. If identity is supported, point mention to existing node.
   c. If evidence indicates a distinct object (e.g. “a second server”),
      create a new referent node even when the head noun repeats.
3. For a pronoun, run RESOLVE; link mention to selected referent or keep
   an ambiguity set.
4. Create discourse unit; infer relation with evidence and attach on right frontier.
5. Generate resolved text by substituting only high-confidence links;
   preserve unresolved forms rather than guessing.

END

Each referent has one node; each mention has a pointer to that node. Re-introduction is a new mention of an existing node; a new indefinite NP with distinguishing modifier (e.g. ‘a second server’) creates a new node. Cataphora requires a bounded look-ahead pass before finalising links. One-anaphora (‘a faster one’) creates a descriptive mention whose head inherits the comparison class but denotes a distinct entity.

A4.3 Demonstration and rewrite rules

For A4 text: R1=lab server; R2=Ravi; R3=second server; R4=room; R5=first-build crash event. S2 ‘It’→R1. S4 ‘it’→R3 (the second server), not R1, because ‘a second server’ introduced a new node in S3. S5 introduces room as a new node. Relations: S1→S2 Explanation/Result (crash of configured server); S3→S4 Sequence; S4→S5 implicit temporal/coherence relation, left UNDECIDED unless context confirms a causal link.

For T: server_1, old_machine_1, Ravi, toolchain_1, suite_1, memory_1, fault_1, vendor_1, module_1. Links: S2 It→server_1; S4 He→Ravi; S4 it→server_1; S5 The suite→suite_1. Relations: S1–S2 Elaboration; S3–S4 Sequence; S4–S5 Result/Explanation; S5–S6 Response/Sequence. Cataphor example: ‘Before he spoke, Ravi checked the server’ → he points forward to Ravi after look-ahead. One-anaphor: ‘A second server arrived; Ravi configured a faster one’ → new server node linked to server class, not same identity. Implicit causal: ‘The memory filled. The build stopped.’ → Result only if domain evidence supports it.

Rewrite rules: (1) replace a repeated full proper name with a pronoun only if exactly one compatible focused antecedent exists; (2) use a definite description when two candidates compete; (3) suppress an emitted sentence only if its proposition is already in the discourse plan with the same arguments, polarity and time; never delete a clarification or commitment.

A4.4 Changed assumptions

Quoted reported speech is a separate attribution context: do not merge speaker beliefs with narrator facts. A section heading starts a new discourse segment and resets or partitions focus; nodes from the previous section are not automatically available. A translated pronoun that loses source gender violates

the assumption that grammatical features are preserved across translation. Keep a cross-language entity ID and mark gender unknown; do not reuse a gender-dependent node link without confidence.

A5 – Coherence, Global Salience and Statistical View

A5.1 Opportunities and safety

Technique	Opportunity in T	Safety condition
Coreference chains	Track server, Ravi and suite across sentences.	Only merge mentions with compatible identity evidence; preserve ambiguity.
Lexical chaining	Server/machine/module and suite/compiler/build help detect topic groups.	Use contextual sense and entity identity; shared words alone do not prove one topic.
Centering	Choose between ‘Ravi’ and ‘he’, ‘server’ and ‘it’.	Use pronoun only when one antecedent is uniquely salient.
Entity grid	Track entity roles (subject/object/other/absent) across sentences for coherence.	Need reliable entity tagging and a representative labelled comparison corpus.
Discourse extraction	Select purchase/configuration/crash as nuclei and supporting details as satellites.	Do not discard safety-critical details such as failure and replaced module.
Distributional similarity	Detect related terms such as compiler/toolchain/build without a hand thesaurus.	Validate domain and language; similarity is not identity or causation.

A5.2 Forward salience dataflow

GEN(u) is the set of entities made available/salient by unit u (introduced or promoted). KILL(u) is the set of entities whose focus is explicitly lost, superseded or outside the chosen discourse boundary. For may-salience, the join at a merge is set union. Initialise entry(ROOT)=∅. For each unit: OUT[u] = GEN[u] ∪ (IN[u] − KILL[u]); IN[u] = union of OUT of predecessors. Iterate to a fixed point. At document start IN=∅; at the end retain OUT of final units. The finite entity universe and monotonic union over a finite lattice guarantee termination.

```
ALGORITHM FORWARD_SALIENCE(graph)
1. For every unit u: IN[u] ← ∅; OUT[u] ← GEN[u]
2. REPEAT
    changed ← FALSE
    FOR each u in topological order:
      newIN ← UNION(OUT[p] for p in predecessors(u))
      newOUT ← GEN[u] UNION (newIN − KILL[u])
      IF newIN ≠ IN[u] OR newOUT ≠ OUT[u]:
        IN[u] ← newIN; OUT[u] ← newOUT; changed ← TRUE
  UNTIL changed = FALSE
3. RETURN IN, OUT
END
```

A5.3 Illustrative fixed-point trace

For the linear six-unit graph, use entity IDs E1=server, E2=old machine, E3=Ravi, E4=toolchain, E5=suite, E6=memory, E7=fault, E8=vendor, E9=module. GEN is cumulative introductions; KILL is empty in this document-level may-analysis. With a linear graph, one topological pass reaches the fixed point; the second pass confirms no change.

Pass	Unit	IN set	OUT set
1	U1	∅	{E1}
1	U2	{E1}	{E1,E2}
1	U3	{E1,E2}	{E1,E2,E3,E4}
1	U4	{E1,E2,E3,E4}	{E1,E2,E3,E4,E5}
1	U5	{E1,E2,E3,E4,E5}	{E1,E2,E3,E4,E5,E6}
1	U6	{E1,E2,E3,E4,E5,E6}	{E1,E2,E3,E4,E5,E6,E7,E8,E9}
2	All	unchanged	unchanged – terminate

A5.4 Backward presupposition analysis

To compute what a reader must already know, use a backward must-analysis: initialise the document exit to the empty set (or explicitly specified known facts), process units in reverse order, and intersect facts required by all successor paths. A unit's requirement is GEN_required ∪ (OUT_required − KILL_known). Intersection at joins retains only facts required on every path. This is conservative because uncertain assumptions are excluded. A summariser may omit implicit background links; a translator may lose recoverable context and produce more explicit, repetitive noun phrases.

PART B – IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

The accompanying ZIP contains the runnable Python prototype and captured console output. The implementation is deliberately rule-based and domain-limited; tables below distinguish reproducible program behaviour from illustrative hand-calculated experiments.

B1 – Implement and instrument the discourse analyser

B1.1 Validation tests

Test	Input	Expected behaviour	Purpose / outcome
T1 multi-paragraph	Two paragraphs; first discusses server, second discusses a compiler.	Paragraph boundary resets focus; no cross-paragraph pronoun link without evidence.	Checks topic shift and hard-boundary handling; prototype sentence splitter requires paragraph handling to be added for full corpus use.
T2 same head noun	'A server failed. A second server arrived. Ravi configured it.'	Create server_1 and server_2; 'it' should link to server_2 if uniquely supported.	Checks identity distinction; domain prototype does not claim general same-head resolution.
T3 boundary	'The server failed.'	One unit, zero pronoun links, no crash.	Checks single-sentence/no-pronoun case; passes.

Gold verification for supplied T uses the four links stated in A1.4. The implementation emits a trace and token/unit counts; the code's small lexicon is not a complete markable detector, so no unsupported corpus-wide F1 is claimed.

B1.2 Design versus implementation

Implementation uses a regular-expression sentence splitter and a hand-built noun-phrase inventory. This makes the supplied examples repeatable without external models. A production version should add paragraph-aware splitting, POS/dependency parsing, explicit pleonastic-it detection for more constructions and annotation import. These are disclosed limitations, not hidden successes.

B1.3 Two salience strategies

Version 1 chooses the most recent compatible entity. Version 2 uses score = 1×recency rank + 3×subject/topic role + up to 3×repetition + 2×parallel role. Agreement is a hard filter. Recency reflects local continuity; grammatical role and repetition approximate prominence; parallelism helps resolve coordinated or repeated clause patterns.

B1.4 Measurement table

The following is a hand-calculated illustrative test-set table (not a measured claim from an external corpus). Gold links = 4. The difficult link is assumed to remain ambiguous when the focus window is large, so the strategy gap disappears when all candidates are retained.

k	V1 P/R/F1	V2 P/R/F1	Mean candidates V1/V2	Unresolved V1/V2	Interpretation
1	0.75 / 0.75 / 0.75	0.75 / 0.75 / 0.75	1.0 / 1.0	1 / 1	Only one candidate survives; weighting cannot help.
2	0.67 / 0.75 / 0.71	0.75 / 0.75 / 0.75	1.5 / 1.5	1 / 1	Role and repetition improve one ambiguous choice.
3	0.67 / 0.75 / 0.71	0.80 / 1.00 / 0.89	2.0 / 2.0	1 / 0	More context helps Version 2 retain a strong antecedent.
Unlimited	0.60 / 0.75 / 0.67	0.60 / 0.75 / 0.67	2.5 / 2.5	1 / 1	Old distractors erase the advantage; better features/abstention are needed.

B2 – Adversarial input

Fixed focus size k=2. Both texts have six sentences and four pronouns.

Text A (easy): (1) Ravi bought a server. (2) Ravi placed the server in the lab. (3) He connected it. (4) The server started. (5) Ravi tested it. (6) He recorded the result.

Text B (hard): (1) Ravi met Arun. (2) The technician spoke to the manager. (3) He thanked him. (4) The engineer arrived. (5) He called him. (6) The supervisor confirmed it.

Prediction before execution: B resolves worse; first uncertain link is S3 ‘He’ because both technician and manager are compatible, and the preceding sentence does not uniquely establish the speaker. B contains multiple same-gender candidates and role competition; A has stable, repeated antecedents. Illustrative predicted results: A = 4/4 correct, 0 unresolved, mean candidate set 1.0; B = 1/4 correct, 1 unresolved, 2 ambiguous, mean candidate set 2.5. Reordering B to introduce a clear subject immediately before each pronoun is expected to change at least two links without changing the set of facts. These are designed predictions; run the analyser and record the actual output before reporting them as measured results.

B3 – Dialogue and conversational agents

Dialog D act labels (simplified): U1 INFORM; A1 REQUEST; U2 INFORM; A2 INFORM + OFFER; U3 ACCEPT + QUESTION; A3 INFORM + COMMIT/RESERVE; U4 THANK/FEEDBACK. The prototype uses single primary labels, so multi-act turns are simplified.

B3.1 Three small dialogues

Dialogue type / turn	Gold act	Predicted act	Response appropriateness
Information: ‘What time does the lab open?’	QUESTION	QUESTION	Appropriate: answer opening time if known.
Clarification: ‘Do you mean the main library?’	CLARIFICATION	QUESTION	Partly appropriate; classifier needs explicit clarification feature.
Repair: ‘Sorry, I meant Friday, not Thursday.’	REPAIR	REPAIR	Appropriate: update date and confirm correction.

The optimized state should preserve the same response decisions as full history for the tested dialogues. Version 1 stores all turn/entity features; Version 2 keeps a recency-ordered stack of active act/entity pairs plus persistent slots for commitments, IDs and dates. This prevents a short window from deleting durable commitments.

B3.2 Conservative and unsafe cases

Conservative case: U asks ‘Is the NLP book available?’ This is a question, not evidence that the book is available or reserved. If the state builder incorrectly marks every mentioned entity as established, the agent may say ‘Your NLP book is reserved’ after the question turn. Correct design records the book as queried, not as available/committed; no reservation transformation is performed.

Unsafe case: U says ‘I need to renew three books’; A asks for the library ID; U supplies it. Dropping A’s clarification turn as redundant can cause the agent to forget why the ID was supplied and may attach it to the wrong service request. Likewise, merging two separate INFORM turns can change the agent’s commitment. Keep clarification and repair acts in the state.

B3.3 Scalability

Illustrative operation model: full-history recomputation processes every prior turn at each new turn, giving approximately n(n+1)/2 turn visits; incremental stack update processes the new turn and active entities, approximately n×w visits for window w. For n=10, 50, 100 turns, full-history visits are 55, 1,275, 5,050; with w=5, incremental visits are 50, 250, 500. These are operation counts from the stated model, not measured wall-clock timings.

B3.4 Trade-off

The optimized representation reduces repeated scanning and keeps commitments accessible. Full history grows quadratically if recomputed from scratch; incremental state is approximately linear for fixed window size plus persistent slots. Assume a 100 ms state-update budget for a small student-facing service. At 100 turns, use the incremental representation; retain full history for audit and rebuild only when a correction or explicit context reset occurs. The threshold must be measured on the deployment hardware.

B4 – Final engineering decision

B4.1 Three processing levels

Level	Components and order	Pronouns before → after	Output / effort	Quality estimate
None	Sentence split → literal translation / direct answer; no document coreference.	4 → 4 unresolved	6 sentences; ~1 pass per token.	Lower adequacy; illustrative chrF 0.45.
Fast (default)	Sentence split → local reference resolution → light discourse plan → surface realisation → target-language translation.	4 → 1 unresolved, 1 ambiguous	6 sentences; ~3 operations per token plus local links.	Illustrative chrF Hindi 0.70, Tamil 0.62; human adequacy: mostly correct, review ambiguous links.
Full (on request)	Paragraph/EDU segmentation → entity graph/coreference → global discourse plan → controlled aggregation → translation via shared interlingua + language-specific realisation.	4 → 0 unresolved, 0 ambiguous on curated T	May add 2–4 analysis passes; ~8 operations per token.	Illustrative chrF Hindi 0.78, Tamil 0.70; human adequacy: meaning preserved on curated test, style still reviewed.

Order matters: resolve references before translation so target pronouns/noun phrases inherit the correct entity; construct the discourse plan before realisation so aggregation and ordering preserve relations; perform language-specific surface realisation after language choice.

B4.2 Counter-example
A full-level system may aggregate two short instructions into one long Hindi sentence, making it harder to follow, or may map an honorific through an interlingua that lacks politeness features. The fast level can be better because it preserves sentence boundaries and avoids unsupported aggregation. Human review should override the full output.

B4.3 Production decision note
Default: Fast, because it improves local reference consistency while meeting a practical latency target. Full runs only when the user requests document-level consistency or when ambiguity is detected and latency permits. Guarantees: preserve source propositions and event order; keep entity IDs consistent; abstain on unresolved references. Non-guarantees: perfect pronoun resolution, best/natural translation, or culturally appropriate style without review.

Second-language cost: separate the language-neutral interlingua from language-specific transfer/surface-realisation rules. Add Tamil morphology, agreement and lexicalisation modules without rebuilding reference resolution and discourse planning. If statistical alignment is language-pair-specific, it must be trained separately; a shared interlingua reduces duplicated analysis.

Skip or limit document-level analysis for short independent notices, strict low-latency requests, noisy OCR, or when confidence is low. Monitor reference-link F1, ambiguity/abstention rate, human adequacy, chrF by language, latency percentiles, edit rate, and errors involving dates, quantities, warnings and commitments before changing the default.

Conclusion
The assignment’s central engineering lesson is that discourse analysis and generation must preserve entity identity, relations and commitments, not merely produce fluent sentences. A practical system should expose uncertainty, measure both local links and downstream adequacy, and use deeper analysis only when its quality gain justifies the additional cost.